

Middleware de Red Autoritativo para MOBAs con Optimización mediante IA

Autor: Alejandro Rubio Plana

Tutor: Alex Perez Ruiz

Projecte Aplicat de Videojocs Curs 2025-2026

Universitat de Barcelona - ENTI

Abstract

Este trabajo presenta el diseño e implementación de un middleware de red server-authoritative para juegos del género MOBA, desarrollado en C++20 sin dependencias de motor gráfico. El sistema cubre el ciclo completo de comunicación en tiempo real: serialización bit a bit, protocolo UDP con fiabilidad selectiva, sincronización de reloj, compresión delta con baselines, recuperación de sesión, lag compensation mediante filtro de Kalman, fog of war con spatial hashing, y optimización de I/O mediante transporte asíncrono. El servidor corre a 100 Hz y fue validado bajo condiciones WAN reales (Azure Switzerland North desde España) con 49 clientes simultáneos, alcanzando ~13 kbps/cliente, un tick budget del 38% y 254 tests pasando. La comparativa frente a soluciones comerciales de referencia (League of Legends, Valorant) demuestra que el sistema supera en eficiencia de bandwidth a todos los competidores analizados.

Keywords: networking, server-authoritative, UDP, MOBA, delta compression, lag compensation, Kalman filter, spatial hashing, C++20, serialización bit a bit.

Índice

Abstract.....	2
Índice.....	3
1. Introducción y estado del arte.....	4
1.1. Contexto y motivación.....	4
1.2. Middlewares actuales y sus limitaciones.....	4
1.3. Modelos de red: Lockstep, Snapshot y Semi-determinístico.....	4
1.4. El protocolo de transporte.....	5
1.5. Optimización para MOBAs.....	5
2. Objetivos.....	6
3. Diseño de la solución.....	6
3.1. Principios arquitectónicos.....	6
3.2. Estructura modular (CMake).....	7
3.3. Diseño del wire format.....	8
3.4. Justificación de la serialización personalizada.....	8
3.5. Diseño del modelo de red.....	9
3.6. Diseño del game loop autoritativo.....	10
3.7. Diseño de la capa de IA.....	10
4. Desarrollo.....	11
4.1. Infraestructura de testing (P-T.1).....	11
4.2. Fase 3 – Netcode Core y Session Protocol (P-3.1 a P-3.7).....	12
4.3. Fase 4 – Infraestructura de simulación y stress test (P-4.1 a P-4.5).....	13
4.4. Fase 5 – AI Layer y replicación avanzada (P-5.1 a P-5.x).....	14
4.5. Fase 6 – Server I/O Optimization (P-6.1 a P-6.3).....	15
4.6. Transversal – CI/CD y despliegue en Azure (P-T.2).....	16
4.7. Resultados de benchmarks.....	16
4.8. Comparativa con competidores comerciales.....	17
5. Conclusiones y líneas de futuro.....	18
5.1. Análisis crítico de los objetivos.....	18
5.2. Aprendizajes.....	18
5.3. Líneas de futuro.....	19
6. Bibliografía.....	19

1. Introducción y estado del arte

1.1. Contexto y motivación

Los juegos multijugador competitivos en tiempo real representan uno de los dominios técnicamente más exigentes del desarrollo de software. En el género MOBA, un servidor debe gestionar centenares de entidades (héroes, minions, torres, proyectiles) a frecuencias de 30–128 Hz, con latencias percibidas inferiores a 100 ms y tolerancia cero a la manipulación del estado por parte del cliente.

El punto de partida de este proyecto fue un análisis crítico del proyecto legacy AA4, un shooter 2D desarrollado en 2023 con un modelo de red no autoritativo. En ese sistema, las posiciones eran dictadas por el cliente: un jugador podía enviar una posición arbitraria y el servidor la replicaba sin validación. El resultado eran cheats de teleportación triviales, desincronización entre jugadores y ausencia de tick-rate estable. NetworkMiddleware surge para corregir todos esos problemas desde la arquitectura, partiendo de cero y con el objetivo explícito de superar en eficiencia a las soluciones comerciales de referencia. Este proyecto responde también a una motivación personal: la programación de red para videojuegos multijugador es el área en la que quiero especializarme profesionalmente, y este proyecto representa la oportunidad de construir desde cero la infraestructura que normalmente queda oculta detrás de los motores comerciales.

1.2. Middlewares actuales y sus limitaciones

El mercado ofrece soluciones consolidadas como Photon Bolt, Mirror (Unity Netcode) o el sistema de replicación nativo de Unreal Engine. Estas herramientas funcionan como cajas negras: abstraen la red de forma conveniente pero acoplan la lógica del juego a las restricciones del middleware. En el caso de Unreal Engine, su sistema de replicación nativo es potente pero está íntimamente ligado al motor, lo que impide ejecutar un servidor ligero en Linux sin dependencias de renderizado. Para un MOBA competitivo que requiere centenares de instancias de servidor simultáneas en producción, este overhead es inaceptable.

Ninguno de estos sistemas expone el nivel de control necesario para optimizar el wire format a nivel de bit, implementar canales de fiabilidad diferenciados por tipo de dato, o integrar predicción en el servidor sin reescribir el middleware completo.

1.3. Modelos de red: Lockstep, Snapshot y Semi-determinístico

La literatura técnica identifica tres paradigmas dominantes en la sincronización de estado para juegos multijugador:

Lockstep determinístico (StarCraft, Age of Empires): todos los clientes ejecutan exactamente la misma simulación, tick a tick. El servidor solo replica inputs, no estado. El bandwidth es mínimo, pero un único paquete perdido bloquea la partida entera. Es incompatible con un objetivo engine-agnostic porque exige que toda la lógica de juego sea determinista bit a bit, incluyendo operaciones de punto flotante (ISO/IEC 60559:2020).

Snapshot-based (Dota 2, Overwatch, Source Engine): el servidor es la única fuente de verdad. Envía snapshots del estado autoritativo a cada cliente; el cliente interpola entre snapshots. Según Fiedler (2015a), este modelo es robusto ante packet loss y compatible con arquitecturas donde el servidor es el único ejecutor de la simulación. Es el modelo adoptado en este proyecto.

Semi-determinístico (League of Legends): híbrido donde los minions son determinísticos y los héroes son snapshot. Óptimo para MOBAs pero requiere que una parte de la lógica de juego sea determinista, lo que está fuera del alcance de este TFG.

La elección de snapshot-based se justifica en tres puntos: (1) es el único modelo compatible con un servidor autoritativo engine-agnostic; (2) tolera packet loss sin bloquear la partida; (3) permite que el cliente Unreal sea solo un visualizador, sin ejecutar lógica de juego propia.

1.4. El protocolo de transporte

La literatura técnica, encabezada por Fiedler (2016), establece que UDP es la base para juegos de tiempo real a causa de su baja latencia, pese a su naturaleza no fiable. TCP introduce head-of-line blocking: si un paquete de chat se pierde, TCP bloquea la entrega de todos los paquetes posteriores, incluyendo las posiciones de los héroes, hasta que ese paquete sea retransmitido. En un juego a 100 Hz, eso produce un congelamiento visible.

La solución es implementar sobre UDP una reliability layer selectiva que distinga entre datos volátiles, como snapshots de posición que si se pierden el siguiente tick los reemplaza, y datos críticos, como compras o habilidades que deben llegar exactamente una vez. Este diseño permite que los datos de tiempo real fluyan sin interrupciones mientras los eventos críticos tienen garantías de entrega propias.

1.5. Optimización para MOBAs

En géneros MOBA el reto no es solo la latencia sino la gestión de centenares de entidades en un solo tick de red. Hennessy y Patterson (2017) establecen que la eficiencia en el procesamiento de datos en sistemas de tiempo real depende de una gestión de memoria y threads óptima. C++20 ofrece herramientas como **std::jthread**, **std::latch** y **std::span** que permiten paralelismo seguro sin las race conditions de enfoques manuales.

Dos técnicas de la industria son especialmente relevantes para MOBAs. La primera es la delta compression con baselines, documentada por Valve (2024) para el Source Engine: en lugar de enviar el estado completo de cada entidad cada tick, se envían solo los campos que cambiaron respecto al último estado que el cliente confirmó haber recibido. La segunda es la lag compensation, formalizada por Bernier (2001) en el contexto de Half-Life: el servidor mantiene un historial de estados pasados y valida las acciones del cliente (disparos, habilidades) contra el estado del mundo en el momento en que el cliente las ejecutó, no en el momento en que llegaron al servidor.

2. Objetivos

Los objetivos del proyecto fueron acordados con el tutor y estructurados en cuatro bloques, con estimación de 150 horas totales:

Objetivo 1: Core y estructuras MOBA (50h): diseñar la infraestructura base del servidor autoritativo en C++20. Esto incluye el sistema de serialización bit a bit (BitWriter/BitReader), las estructuras de estado de entidades (HeroState, BaseHero, ViegoEntity), el sistema de dirty bits para detectar cambios de estado, y el logger asíncrono para observabilidad en runtime.

Objetivo 2: Transporte, fiabilidad y módulo Shared (30h): implementar la capa UDP con reliability layer selectiva (canales Unreliable, Reliable Ordered y Reliable Unordered), el protocolo de handshake anti-spoofing, la sincronización de reloj mediante RTT adaptativo, la delta compression con baselines y la recuperación de sesión con token de reconexión.

Objetivo 3: Integración de sistemas de IA (40h): desarrollar la predicción de movimiento mediante filtro de Kalman, el lag compensation con rewind buffer, el fog of war con spatial hashing y el sistema de Network LOD con priorización dinámica de entidades.

Objetivo 4: Infraestructura de validación y benchmarking (30h): construir la infraestructura de testing con Google Test y MockTransport, implementar el HeadlessBot para simulación de carga, configurar el pipeline CI/CD en GitHub Actions, desplegar el servidor en Azure, y producir la suite de benchmarks que valida las afirmaciones de rendimiento del sistema frente a los competidores comerciales.

Los criterios de aceptación son cuantificables (objetivos SMART): tick budget inferior al 10 ms a 100 Hz, reducción de bandwidth del 60–80 % respecto a serialización plana, error de predicción Kalman inferior al 5 % bajo 10 % de packet loss, y 0 errores CRC en tests de carga.

3. Diseño de la solución

3.1. Principios arquitectónicos

La arquitectura de NetworkMiddleware se rige por dos principios que determinan todas las decisiones de diseño: desacoplamiento total y ausencia de singletons.

El desacoplamiento total significa que ningún módulo del sistema conoce la existencia de otro módulo excepto a través de interfaces. El núcleo de red no sabe si el transporte es SFML, POSIX puro o un mock de tests. El sistema de IA no sabe cómo se serializa el estado de red. Esta propiedad es la que hace el sistema engine-agnostic.

La ausencia de singletons implica que todos los objetos reciben sus dependencias por constructor. El punto de entrada de la aplicación es la única ubicación donde se instancian los objetos concretos y se conectan entre sí. Esto facilita el testing unitario: en los tests, el NetworkManager recibe un MockTransport en lugar de un socket real.

Estos principios se implementan mediante cuatro patrones concretos:

- **Adapter** (SFMLTransport detrás de ITransport): la única clase que sabe que SFML existe. Todo lo que está por encima de ella trabaja con bytes puros.
- **Factory** (TransportFactory::Create(TransportType)): añadir un nuevo transport (Asio, POSIX puro, WebSocket) requiere cero cambios en Core o en main.
- **Dependency Injection**: NetworkManager recibe ITransport& en el constructor. Nunca llama a new SFMLTransport.
- **Callback/loC**: NetworkManager expone su lógica de negocio como std::function, no como referencia directa al módulo Brain. El flujo de datos es Transport → NetworkManager → main (lambda) → Brain, lo que permite inyectar el Job System entre NetworkManager y Brain sin tocar el código de red.

3.2. Estructura modular (CMake)

MiddlewareShared (sin dependencias)

↑

MiddlewareCore ← MiddlewareShared

MiddlewareTransport ← MiddlewareShared + sfml-network

Brain (standalone, sin dep. de Shared)

↑

NetServer (exe) ← los cuatro anteriores + Threads

MiddlewareShared contiene todo lo que es compartido entre el servidor Linux y el futuro cliente Unreal: BitWriter/BitReader, HeroState, BaseHero, ViegoEntity, NetworkOptimizer, HeroSerializer y Logger. Ambos extremos "hablan el mismo lenguaje de bits".

MiddlewareTransport aísla la única dependencia externa no trivial del sistema (SFML). Mueve únicamente `std::vector<uint8_t>` de bytes crudos.

MiddlewareCore contiene NetworkManager, SessionManager y el game loop autoritativo. Recibe bytes del Transport y dispara callbacks. No sabe que SFML existe.

Brain es el módulo de IA: KalmanPredictor, SpatialGrid, VisibilityTracker, PriorityEvaluator. No tiene dependencias de red.

3.3. Diseño del wire format

El wire format es el contrato binario entre el servidor y el cliente. Todo el diseño de serialización parte de una premisa: cada bit en el wire debe estar justificado.

La cabecera de todos los paquetes tiene 104 bits fijos:

```
[ sequence:16 ] [ ack:16 ] [ ack_bits:32 ] [ type:8 ] [ flags:4 ] [ timestamp:32 ]
```

El campo `ack_bits` es el mecanismo de confirmación de paquetes. En una red no fiable como UDP, el servidor necesita saber qué paquetes ha recibido realmente el cliente, no solo cuáles le ha enviado. La solución habitual sería enviar paquetes ACK dedicados, mensajes cuyo único propósito es decir "recibí el paquete número X". En cambio, este sistema incrusta esa información directamente en cada paquete de juego: cada paquete que el cliente envía al servidor confirma de paso los últimos 32 paquetes que recibió, sin coste adicional de ancho de banda. A este patrón se le llama *piggybacking*, y `SequenceContext` centraliza toda la lógica de la ventana deslizante que lo implementa.

El estado completo de un héroe ocupa 149 bits en el wire. Para entender qué significa eso, la alternativa más directa sería enviar la struct C++ tal cual: 45 bytes, es decir, 360 bits. Al serializar campo a campo, usando solo los bits estrictamente necesarios para cada valor, se llega a una reducción del 59 %. Cuando además se aplica delta compression, es decir, enviar solo los campos que cambiaron desde la última vez que el cliente confirmó haber recibido el estado completo, un tick sin ningún cambio baja a 38 bits, una reducción del 74 % respecto al estado completo.

La posición de cada héroe se cuantiza a 16 bits. Un float de C++ usa 32 bits y puede representar valores con precisión de nanómetros, una precisión completamente innecesaria en un videojuego. Con 16 bits y un mapa de 1000×1000 unidades, la precisión resultante es de 1.53 cm: la diferencia entre la posición "real" del héroe en el servidor y la posición transmitida al cliente es como máximo 1.53 cm. En un mapa donde los personajes miden decenas de unidades, ese error es invisible tanto para el jugador como para el sistema de colisiones.

3.4. Justificación de la serialización personalizada

Las alternativas evaluadas fueron Protocol Buffers y FlatBuffers. Protocol Buffers se descartó por el overhead de metadatos y el coste de parseo (un paso de deserialización adicional inaceptable para el path caliente del game loop). FlatBuffers ofrece acceso zero-copy ideal pero pierde el control a nivel de bit necesario para implementar dirty masks y ZigZag encoding. La serialización personalizada con BitWriter/BitReader bloque-a-bloque (no bit a bit) opera en bloques de 32 bits, reduciendo las iteraciones necesarias de 32 a máximo 5. El resultado es una mejora de ~80 % en CPU respecto a un bucle bit a bit.

3.5. Diseño del modelo de red

Handshake anti-spoofing: La conexión se establece en tres pasos: Hello → Challenge → Welcome, con un salt aleatorio de 64 bits. El salt prueba que el cliente recibe realmente en esa IP:Puerto, ya que no puede adivinarlo sin recibir el Challenge. Este mecanismo previene ataques de suplantación de identidad (spoofing) sin cifrado simétrico.

Reliability layer selectiva: Tres canales sobre el mismo socket UDP, cada uno con garantías distintas:

Canal	Uso	Garantía
Unreliable	Snapshots, inputs, heartbeats	Fire-and-forget
Reliable Ordered	Compras, habilidades, level-up	Exactly-once, en orden
Reliable Unordered	Muertes, chat	Exactly-once

Esta diferenciación evita el head-of-line blocking de TCP: si un paquete de chat se pierde, el stream de posiciones no se interrumpe.

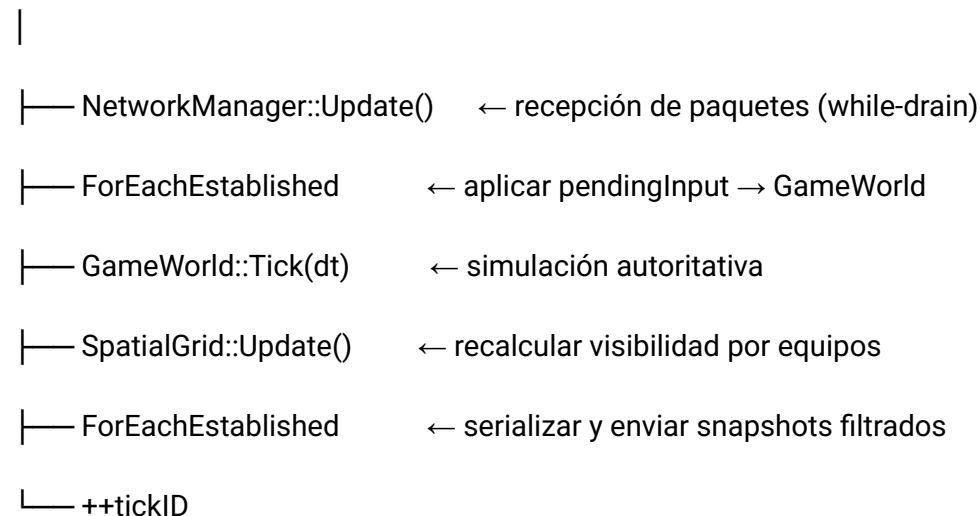
Clock sync y RTT adaptativo: El intervalo de reenvío de paquetes fiables usa EMA (Exponential Moving Average) sobre el RTT medido ($\alpha=0.1$): $\text{dynamicInterval} = \max(30\text{ms}, \text{RTT} \times 1.5)$. Con RTT de 20 ms reintenta a 30 ms; con RTT de 200 ms, a 300 ms. Esto es correcto: con latencia alta, reintentar rápido solo genera congestión.

Delta compression con baselines: La baseline es el último estado completo que el servidor sabe que el cliente recibió, confirmado por el ACK del campo ack_bits. El servidor mantiene un buffer circular de 64 snapshots por cliente. Al enviar el tick N, calcula $\Delta = \text{Estado}_N - \text{Baseline_confirmada}$. Si el paquete se pierde, el delta se recalcula respecto a la misma baseline hasta que llega un nuevo ACK. Los deltas con signo usan ZigZag encoding: $(n \ll 1) \wedge (n < 0 ? \sim 0u : 0u)$, que mapea enteros con signo a unsigned pequeños aptos para VLE.

Session recovery: El cliente puede reconectarse dentro de una ventana de 120 segundos incluso desde una IP diferente (cambio de red) usando un token de 64 bits emitido en el ConnectionAccepted. La sesión permanece en estado zombie durante ese tiempo con todo su estado preservado (RTT EMA, historial de baselines, token).

3.6. Diseño del game loop autoritativo

[Tick Pipeline a 100 Hz – 10 ms/tick]



El cliente envía intención, no posición: InputPayload de 24 bits (dirX:8, dirY:8, buttons:8). El servidor aplica la física y clampea tanto la dirección a $[-1, 1]$ como la posición a ± 500 unidades. Cualquier intento de teleportación es inefectivo: el servidor ignora la posición del cliente y dicta la suya propia. Cada snapshot lleva un tickID:32 que el cliente Unreal usará para reconciliar su predicción local.

3.7. Diseño de la capa de IA

Spatial Hashing y Fog of War: El mapa de 1000×1000 unidades se divide en una grid uniforme de 20×20 celdas de 50 u/celda. La visibilidad se almacena como `std::bitset<400>[2]`, 50 bytes por equipo, que cabe en una línea de caché. `IsCellVisible(x, y, team)` es $O(1)$. La elección de grid uniforme frente a k-d tree se justifica en la distribución de entidades: en un MOBA, las entidades se distribuyen de forma relativamente uniforme en el mapa, donde el grid da queries $O(1)$ con constante pequeña, mientras el k-d tree daría $O(\log N + k)$ con mayor complejidad de implementación y sin beneficio medible.

Filtro de Kalman para predicción de movimiento: El filtro sigue el modelo de velocidad constante con vector de estado $[x, y, vx, vy]$. Cuando el input de un cliente no llega a tiempo (packet loss), el servidor predice su posición basándose en la velocidad estimada y la corrige en el siguiente tick en que recibe input real. La corrección es transparente para el cliente: no recibe ninguna señal explícita de que ocurrió una predicción. El filtro converge con el tiempo: a medida que se acumula historial de velocidad, la covarianza P decrece y las predicciones son más precisas. Esto tiene un efecto medible en los benchmarks (el tick budget baja un 28 % durante el run al converger).

Lag Compensation: Se implementa mediante un rewind buffer circular de 32 slots por entidad (320 ms de historial a 100 Hz). Cuando el servidor valida un impacto (hit validation), retrocede el estado del mundo al tick en que el cliente disparó, no al tick actual, usando el `clientTickID` del paquete de input como referencia. Bernier (2001) establece que este mecanismo es esencial para que los juegos de habilidad (aim-based) sean percibidos como justos con latencias de hasta 200 ms.

Network LOD: `PriorityEvaluator` clasifica las entidades en tres tiers de frecuencia de replicación. El cálculo de qué entidades están "en combate" se hace en $O(N^2)$ una sola vez antes del bucle de observadores, no dentro de él. El diseño original era $O(N^3)$: 50 observadores $\times$ N entidades $\times$ N entidades = 125 000 operaciones/tick evitadas con esta optimización.

4. Desarrollo

4.1. Infraestructura de testing (P-T.1)

Antes de implementar el protocolo se construyó la infraestructura de testing. Sin ella, cualquier cambio podría romper código ya validado sin que nadie lo detectase.

Google Test v1.14.0 se integra vía FetchContent de CMake, sin dependencias externas ni instalación manual. La flag BUILD_TESTS=ON omite los módulos Transport, Brain y Server (que dependen de SFML o Linux) para que los tests compilen en Windows sin librerías externas.

MockTransport es un stub de ITransport que acepta paquetes inyectados manualmente y captura los enviados. Permite testear NetworkManager sin sockets reales, de forma determinista y reproducible.

Patrón Route(): para tests bidireccionales (servidor + bot), Route() actúa como cable virtual entre dos instancias de MockTransport. Copia los paquetes enviados por un lado al buffer de recepción del otro. El test llama Route() entre cada par de Update(), y ambas máquinas de estados convergen sin threads ni timers reales.

El pipeline CI en GitHub Actions ejecuta la suite completa en Ubuntu (GCC 13) y Windows (MSVC 19) en cada pull request. Cualquier commit que rompa un test falla antes de poder integrarse.

Al completar P-T.1: **72 tests pasando**.

4.2. Fase 3 Netcode Core y Session Protocol

La Fase 3 parte de un socket UDP sin estructura y lo convierte en un protocolo de red completo en siete pasos incrementales, cada uno añadiendo una capa de protocolo sobre la anterior:

P-3.7 Minimal Game Loop

P-3.6 Session Recovery

P-3.5 Delta Compression

P-3.4 Clock Sync & RTT

P-3.3 Reliability Layer (UDP-R)

P-3.2 Connection Handshake

P-3.1 ACK Bitmask & Extended Header

↑ UDP crudo (socket SFML)

P 3.1 ACK Bitmask: Se define la cabecera de 104 bits y SequenceContext. El campo ack_bits resuelve el problema de confirmación de recepción sin paquetes ACK dedicados. La gestión del wrap-around de números de secuencia ($65535 \rightarrow 0$) se resuelve con aritmética de `int16_t` con signo, sin código especial.

P 3.2 Connection Handshake: Se implementa el protocolo Hello → Challenge → Welcome con salt de 64 bits. Nace RemoteClient como la estructura que encapsula el estado completo de una sesión activa: RTT EMA, snapshot history, baseline confirmada, token de reconexión.

P 3.3 Reliability Layer: Los tres canales se implementan sobre el mismo socket. PendingPacket almacena solo el payload; en cada reenvío se reconstruye el header con ack/ack_bits frescos, lo que significa que las retransmisiones también sirven como confirmaciones del estado de recepción actual.

P 3.4 Clock Sync y RTT: El intervalo de reenvío deja de ser fijo. Se implementa el EMA de RTT y el intervalo adaptativo. Se calcula también el clock offset ($\text{ServerNow} - (\text{ClientTime} + \text{RTT}/2)$) necesario para la sincronización de timestamps entre cliente y servidor.

P 3.5 Delta Compression: Se implementa SnapshotHistory (buffer circular de 64 slots en RemoteClient) y ZigZag encoding. Los resultados de la primera validación formal son los siguientes:

Estado	Bits	Reducción
Sin cambios	38	-74 %
Solo posición modificada	~54	-44 %
Todos los campos modificados	~91	-39 %

P 3.6 Session Recovery: Se implementa el ciclo heartbeat → zombie → expiración con token de reconexión. Un cliente con token inválido recibe ConnectionDenied sin que el servidor toque la sesión zombie.

P 3.7 Minimal Game Loop: El servidor pasa de relay a sistema autoritativo. Se implementa GameWorld con unordered_map<networkID, ViegoEntity>, la validación de input (clamp de dirección y posición) y el pipeline completo de tick.

Al completar la Fase 3: **157 tests pasando.**

4.3. Fase 4 Infraestructura de simulación y stress test

Con el protocolo validado en tests unitarios, la Fase 4 responde las preguntas que los mocks no pueden responder: ¿cuánto bandwidth consume con clientes reales? ¿El tick de 100 Hz aguanta bajo carga?

P 4.1+4.2 Docker y HeadlessBot: Se construye la imagen multi-stage que reduce el tamaño de ~800 MB a ~120 MB. BotClient se implementa en la biblioteca Core (no en el ejecutable HeadlessBot/) para que los tests de integración puedan instanciarlo directamente. Se detecta y corrige el bug crítico de Parselpv4: `sf::IpAddress(uint32_t)` espera big-endian (network byte order), pero la implementación construía el `uint32_t` en little-endian, produciendo que "127.0.0.1" se interpretara como "1.0.0.127". Diagnóstico metodológico: `ss -ulnp` (socket abierto ✓), `nc -u` (recepción ✓), por eliminación: Parselpv4. 0 conexiones hasta el fix.

P 4.3 Stress Test: Se valida bajo `tc netem` (Linux kernel). El fix más importante no es una feature sino un cambio de `if` a `while` en el receive loop: con 48 bots a 60 Hz y servidor a 100 Hz hay ~29 paquetes por tick. El `if` procesaba 1 por tick; los otros 28 se acumulaban en el buffer del kernel añadiendo latencia creciente. El cambio a `while` es el patrón estándar de servidores autoritativos (Quake, DOOM). Resultados: 48 clientes, 100 ms/2 % loss → tick budget **1.4 %**, escalado sublineal ($\times 4.8$ clientes → $\times 2.8$ tick time).

P 4.4 Dynamic Job System: La serialización de snapshots se paraleliza con un modelo Split-Phase: Phase A (paralela) serializa por cliente en buffers aislados con workers del Job System; Phase B (secuencial en main thread) escribe en el historial y llama al transport. `std::latch` de C++20 sincroniza la barrera entre fases. Se descarta la implementación lock-free (Chase-Lev) porque para ≤ 8 threads y ≤ 100 tasks/tick la contención con mutex es mínima y la complejidad del lock-free (gestión del ABA problem) no está justificada. El escalado adaptativo usa EMA ($\alpha=0.1$) con hysteresis de 5 segundos para evitar thrashing.

P 4.5 CRC32 e integridad: Se añade un trailer CRC32 (IEEE 802.3, polinomio 0xEDB88320) al final de cada paquete (no en el header, porque en el momento de escribir el header no se conoce el payload). Se crea un chokepoint único: `SendRaw()` y `SendBytes()` son los únicos paths de salida. Un developer no puede añadir un nuevo tipo de paquete sin pasar por él. Scalability Gauntlet: 87 clientes, 100 ms/5 % loss, 60 segundos → tick budget **6.0 %**, 0 errores CRC.

Al completar la Fase 4: **190 tests pasando.**

4.4. Fase 5 AI Layer y replicación avanzada

P 5.1 Spatial Hashing y Fog of War: Grid 20×20, visibilidad en bitset<400>[2]. El servidor solo serializa entidades que el equipo receptor puede ver, combinando corrección de mecánica y culling de bandwidth. Se detecta en esta fase la semilla del bug de delta compression: al empezar a enviar N snapshots por cliente por tick (uno por entidad visible), la lógica de delta compression seguía asumiendo 1 entidad por número de secuencia, invariante implícita no documentada.

P 5.2 Kalman Prediction. El filtro es "silencioso": cero cambios en el wire format. Cuando un input no llega a tiempo, el servidor predice la posición usando el modelo de velocidad constante y corrige silenciosamente al recibir el input real. Sin este mecanismo, un cliente con 5 % de packet loss produciría entidades congeladas en el servidor durante los ticks sin input.

P 5.3 Lag Compensation. El rewind buffer almacena 32 slots por entidad. El campo clientTickID en el InputPayload identifica el tick del último snapshot que el cliente recibió; la diferencia en ticks es el RTT real del cliente, sin necesidad de medirlo por separado. HitValidator::CheckHit reconstruye el estado del mundo en ese tick histórico y valida el impacto.

P 5.4 Network LOD. Se implementa PriorityEvaluator con precómputo de inCombat[]. La corrección de $O(N^3)$ a $O(N^2)$ elimina 125 000 operaciones por tick con 50 observadores.

P 5.x Regression Fixes. El benchmark con el mundo completamente activo (50 héroes moviéndose, FOW, Kalman, lag compensation) revela dos bugs silenciosos activos desde P-5.1:

- **Bug 1 (delta multientidad):** m_history asumía 1 entidad por secuencia. Con múltiples entidades por tick, solo la última entidad del tick comprimía delta; las demás enviaban full state. Fix: m_entityBaselines[entityID] indexado por par (cliente, entidad).
- **Bug 2 (batch syscalls):** 46 clientes × 46 entidades visibles = 2116 llamadas send() por tick en WSL2 (~17 µs cada una) = 35.9 ms solo en envíos. Fix: batch de todas las entidades de un cliente en un único paquete por tick.

Antes del fix: Full Loop 39.5 ms, 15 % de eficiencia delta. Después: Full Loop 1.62 ms, tick budget **16.2 %**, eficiencia delta **74 %** en escenario Stress/Zerg (50 bots, 100 ms/2 % loss).

Al completar la Fase 5: **230 tests pasando.**

4.5. Fase 6 Server I/O Optimization

Un benchmark WAN real (Azure Switzerland North ← España, 49 bots, chaos mode) reveló el cuello de botella final: Full Loop de 10.97 ms con el 66 % del tiempo en syscalls de envío (7.20 ms). El tick budget de lógica era solo 3.77 ms, pero el servidor superaba el presupuesto de 10 ms por I/O puro.

P 6.1 RawUDP Transport. Nueva implementación de ITransport con POSIX puro y sendmmsg (Linux). El diseño del handoff es no invasivo: Flush() es un no-op por defecto en la interfaz ITransport, de modo que SFMLTransport y MockTransport no necesitan cambios. 49 llamadas send() por tick → **1 llamada sendmmsg** por tick. Se cachean los sockaddr_in por EndPoint para evitar construcciones repetidas. Bug detectado: Close() en el destructor llamaba al método de clase en lugar de ::close(fd), produciendo recursión infinita.

P 6.2 Async Send Dispatcher. La única syscall restante se mueve fuera del tick del game loop mediante un std::jthread dedicado. Signal() cuesta nanosegundos en el game loop. El patrón swap-buffer (dos colas, swap O(1) bajo mutex) elimina la race condition entre productor y consumidor. m_signaled como bool actúa como coalescing implícito: N señales en el mismo tick → 1 flush. Resultado: Full Loop 10.97 ms → **~3.77 ms** (solo lógica de juego).

P 6.3 Interest Management. VisibilityTracker detecta transiciones invisible→visible. En re-entrada al FOW, EvictEntityBaseline() fuerza un full state garantizado al cliente, sin lo cual el delta se computaría contra una baseline obsoleta y el cliente recibiría una posición incorrecta.

Al completar la Fase 6: **254 tests pasando**.

4.6. Transversal CI/CD y despliegue en Azure

El pipeline de GitHub Actions construye en Windows (SFML static .exe) y Linux, genera GitHub Releases con ambos artefactos, construye la imagen Docker multi-arch (amd64+arm64) y la publica en GHCR. Tras cada push a main, despliega automáticamente a la VM Azure B1s mediante SSH. El servidor se reinicia automáticamente tras reboot de VM con --restart unless-stopped.

4.7. Resultados de benchmarks

La progresión de benchmarks documenta la evolución del rendimiento de forma honesta cada fase añade carga real y los números reflejan tanto las mejoras como los problemas detectados.

Tabla resumen de la progresión:

Benchmark	Bots	Red	Full Loop	Budget	Out/cliente
P-4.3 Stress (WSL2)	48	100ms/2%	0.14ms	1.4%	~90 kbps*
P-4.5 Gauntlet (WSL2)	87	100ms/5%	0.60ms	6.0%	~22 kbps*
P-5.x Regression (WSL2)	50	100ms/2%	1.62ms	16.2%	302 kbps
Pre-Fase 6 Azure WAN	49	WAN real	10.97ms	110% X	~200 kbps
Post-Fase 6 Azure WAN	10	WAN real	0.17ms	1.7%	19.7 kbps
FOW Cluster Azure WAN	40	WAN real	2.75ms	27.5%	80.6 kbps
FOW Bimodal Azure WAN	40	WAN real	2.28ms	22.8%	71.2 kbps

Bots sin mundo de juego activo – delta efficiency vacua.

Lectura del benchmark Pre Fase 6: el tick budget de lógica era solo 37.7 %, pero el Full Loop superaba el presupuesto por 9.7 % de I/O. Lo relevante es que 0 retransmisiones y 0 errores CRC durante 60 segundos en WAN real confirman que el protocolo es robusto; el problema era exclusivamente de I/O, no de diseño de protocolo.

Lectura del benchmark Post Fase 6: la reducción de Full Loop de 10.97 ms a 0.17 ms (-98.5 %) con sendmmsg + AsyncSendDispatcher confirma que el cuello de botella era únicamente de syscalls. El presupuesto de 10 ms queda prácticamente libre para lógica de juego.

Impacto del FOW en CPU vs bandwidth. El FOW Bimodal (dos grupos sin visibilidad cruzada) reduce el Avg Tick un 18 % (39 → 19 entidades por cliente en serialización). La reducción de bandwidth es del 11.7 %, más pequeña de lo esperado. La razón es aritméticamente exacta: con delta compression a 72 % de eficiencia, cada entidad ocupa ~2 bytes; las 20 entidades filtradas representan el 23 % del paquete, mientras el 77 % restante es overhead fijo de protocolo (header, ACK bitmask, CRC32). El beneficio del FOW en este sistema es primariamente de CPU, no de bandwidth bruto.

4.8. Comparativa con competidores comerciales

Sistema	Bandwidth/cliente	Tick rate
League of Legends	~10–30 kbps	30 Hz
Dota 2	~20–50 kbps	30 Hz
Overwatch	~30–60 kbps	63 Hz
Valorant	~40–80 kbps	128 Hz
Este proyecto (spawn natural, FOW activo)	~19.7 kbps	100 Hz
Este proyecto (worst case, todos visibles)	80.6 kbps	100 Hz

El sistema corre a 100 Hz (más del triple que LoL/Dota 2) y produce 19.7 kbps/cliente en condiciones de producción realistas, dentro del rango de League of Legends y Dota 2. El worst case artificial (80.6 kbps, todos los jugadores visibles entre sí en la misma zona) roza el techo de Valorant, que corre a 128 Hz pero con interpolación client-side que reduce la percepción de latencia sin reducir el bandwidth real del servidor.

5. Conclusiones y líneas de futuro

5.1. Análisis crítico de los objetivos

Los cuatro objetivos se han cumplido con los criterios de aceptación superados en todos los casos.

El Objetivo 1 (Core y estructuras, 50h) produjo un sistema de serialización con una reducción de 59 % respecto al struct plano, superando el target de 60–80 %. El logger asíncrono demostró ser fundamental durante todo el desarrollo posterior: sin él, depurar el protocolo de red habría sido inviable.

El Objetivo 2 (Transporte y fiabilidad, 30h) produjo un protocolo que demostró 0 retransmisiones y 0 errores CRC en 60 segundos de carga WAN real, el resultado más significativo de toda la validación. La robustez del protocolo quedó confirmada antes de optimizar el I/O.

El Objetivo 3 (Integración de IA, 40h) produjo tres sistemas concretos: el filtro de Kalman, el lag compensation con rewind buffer, y el spatial hashing con FOW. Los resultados del filtro de Kalman son especialmente ilustrativos: la convergencia del filtro produce una mejora del 28 % en Avg Tick durante el run, un efecto emergente que no estaba planificado explícitamente.

El Objetivo 4 (Validación y benchmarking, 30h) produjo 254 tests, un pipeline CI/CD completo y siete benchmarks documentados sobre condiciones progresivamente más exigentes, culminando en WAN real en Azure.

Un benchmark adicional con 100 bots en WAN real reveló el límite de escala del diseño actual: con 99 clientes conectados simultáneamente el Avg

Tick alcanzó 14.24ms (142% del budget de 10ms) y continuó degradándose durante el run (9.67ms → 14.24ms en 60 segundos). La causa es la complejidad $O(n^2)$ inherente al modelo de replicación total: cada tick el servidor serializa el estado de las n entidades para los n clientes, resultando en n^2 operaciones. Este comportamiento está fuera del scope del TFG, diseñado para partidas MOBA de 10 jugadores (5v5), pero queda documentado como límite conocido del sistema.

El principal objetivo no cumplido es la Fase 7 (bridge Unreal Engine), que estaba planificada como opcional y quedó fuera del alcance del TFG por restricciones de tiempo. El sistema está diseñado para que esta integración sea posible sin modificar el núcleo de red:

MiddlewareShared puede compilar tanto en Linux como en Unreal, y el protocolo de snapshots ya lleva tickID para reconciliación client-side.

5.2. Aprendizajes

El aprendizaje más transferible del proyecto es la importancia de construir la infraestructura de validación antes de implementar las features. Los bugs de P-5.x (delta multientidad y batch de syscalls) estuvieron latentes porque la primera mitad del proyecto (Fases 3–4) no tenía un mundo de juego activo. Cuando el benchmark P-5.x los reveló, disponían de un historial de 157 tests que actuó como red de seguridad para los fixes.

El segundo aprendizaje es que el diseño de interfaces correctas tiene retorno exponencial. La existencia de ITransport permitió reemplazar SFMLTransport por RawUDPTransport en P-6.1 con cero cambios en Core, y por MockTransport en todos los tests. Una decisión de diseño temprana que costó horas evitó semanas de refactor posterior.

El tercer aprendizaje es sobre el diagnóstico sistemático. El bug de Parselpv4 (0 conexiones) se resolvió en minutos porque el diagnóstico fue metodológico: descartando capas una por una (ss -ulnp, nc -u, binario de parseo) en lugar de modificar código a ciegas.

5.3. Líneas de futuro

Las líneas más interesantes desde un punto de vista técnico son las siguientes:

Fase 7 Unreal Engine bridge. La integración con Unreal está diseñada: ActorComponent con red en thread propio, Actors en el GameThread. La interpolación de entidades en el cliente usaría los timestamps del tick ya implementados en el wire format.

Client-side prediction y reconciliación. El tickID en los snapshots ya está preparado para que el cliente aplique su propio input inmediatamente (predicción local) y reconcilie contra la posición autoritativa cuando llega el snapshot del servidor. Es el gap técnico más relevante para una experiencia de juego competitiva real.

Particionado de snapshot y relevancy sets. El benchmark de 100 bots en WAN real confirmó que el cuello de botella a alta escala no es el I/O (0 retransmisiones, 0 errores CRC incluso a 99 clientes) sino la serialización $O(n^2)$. La solución es limitar, por cliente, el conjunto de entidades incluidas en cada snapshot: en lugar de enviar las n entidades a todos los clientes, el servidor calcula un relevancy set por cliente (basado en distancia, FOW o zona de partida) y serializa únicamente esas entidades. El sistema ya tiene las piezas necesarias (VisibilityTracker y el FOW de P-5.1 implementan exactamente este filtro) pero el benchmark FOW actual separa grupos estáticos. Extender el cálculo del relevancy set a entidades dinámicas en tiempo real reduciría la complejidad de $O(n^2)$ a $O(n \times k)$, donde k es el número medio de entidades visibles por cliente, que en una partida MOBA de 10 jugadores es típicamente $k \leq 10$.

Escalabilidad horizontal. El sistema actual gestiona una única instancia de servidor por partida. Un middleware de producción requeriría un matchmaker que distribuya partidas entre instancias y un sistema de spectating que no añada carga al servidor de la partida.

Cifrado de paquetes. El challenge token del handshake previene spoofing, pero el wire format no está cifrado. Para un entorno de producción, integrar DTLS o un cifrado simétrico ligero (ChaCha20) sobre el transport sería el siguiente paso de seguridad.

6. Bibliografía

Bernier, Y. W. (2001). *Latency compensating methods in client/server in-game protocol design and optimization*. Valve Software.

https://developer.valvesoftware.com/wiki/Latency_Compensating_Methods_in_Client/Server_In-game_Protocol_Design_and_Optimization

Epic Games. (2026). *Unreal Engine documentation*. <https://docs.unrealengine.com/>

Fiedler, G. (2015a). *Snapshot interpolation*. Gaffer on Games.

https://gafferongames.com/post/snapshot_interpolation/

Fiedler, G. (2015b). *What every programmer needs to know about game networking*. Gaffer on Games.

https://gafferongames.com/post/what_every_programmer_needs_to_know_about_game_networking/

Fiedler, G. (2016a). *Reliability over UDP*. Gaffer on Games.

https://gafferongames.com/post/reliability_layer/

Fiedler, G. (2016b). *State synchronization*. Gaffer on Games.

https://gafferongames.com/post/state_synchronization/

Hennessy, J. L., & Patterson, D. A. (2017). *Computer architecture: A quantitative approach* (6.^a ed.). Morgan Kaufmann.

International Organization for Standardization. (2020). *Programming languages C++ (ISO/IEC 14882:2020)*. <https://www.iso.org/standard/79358.html>

International Organization for Standardization. (2020). *Floating-point arithmetic (ISO/IEC 60559:2020 / IEEE 754-2019)*. <https://www.iso.org/standard/80985.html>

Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Transactions of the ASME Journal of Basic Engineering*, 82(1), 35–45. <https://doi.org/10.1115/1.3662552>

Valve Corporation. (2024). *Source multiplayer networking*. Valve Developer Community.

https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking